



UNIVERSIDAD DE LA CUENCA DEL PLATA
Facultad de Ingeniería y Tecnología — Sede Corrientes
Ingeniería en Sistemas de Información

Simulador de procesos y memoria

Informe técnico — Actividad de Evaluación 2 (intercátedra)
Paradigmas y Lenguajes de Programación II · Sistemas Operativos

Alumna	Valentina Carruega Lacour
DNI	46.921.882
Docentes de Sistemas Operativos	Mirta Luque · Patricia Daniela Montiel
Docente de Paradigmas II	José A. Fernández
Repositorio	github.com/valenlacour/simulador-de-memoria
Versión evaluada	tag <code>v1.0</code>
Fecha	5 de octubre de 2026

Índice

1. Introducción
2. Objetivos
3. Escenario y alcance
4. Descripción general de la solución
5. Diseño: clases y responsabilidades
6. Decisiones de programación orientada a objetos
7. Estructuras de datos utilizadas
8. Estados de los procesos y diagrama de transición
9. Orden de cada tick y planificación Round Robin
10. Gestión de memoria: políticas de asignación y coalescencia
11. Diagramas de secuencia
12. Matriz de trazabilidad RF → clase → pruebas
13. Casos de prueba y resultados
14. Cobertura de código
15. Análisis comparativo First-Fit, Best-Fit y Worst-Fit
16. Análisis de la fragmentación externa
17. Conclusiones
18. Bibliografía y fuentes
19. Anexo: cómo reproducir los resultados

1. Introducción

Un sistema operativo administra recursos escasos que muchos procesos quieren usar al mismo tiempo. Los dos más importantes son la **memoria principal**, que cada proceso necesita para poder ejecutarse, y la **CPU**, que en este escenario es una sola y tiene que repartirse por turnos.

Este trabajo implementa una **biblioteca de clases en TypeScript** que simula ese reparto en unidades discretas de tiempo (*ticks*). Los procesos piden un bloque contiguo de memoria, esperan si no hay lugar, pasan a la cola de Listos, usan la CPU con **Round Robin**, pueden bloquearse por una operación de Entrada/Salida y, al terminar, devuelven su memoria, que se une con los huecos vecinos (**coalescencia**).

La biblioteca no tiene interfaz gráfica, menú ni función `main`. Todo su comportamiento se verifica con **316 pruebas automatizadas** (Vitest) que pasan en su totalidad y cubren el **100 %** del código de producción. El informe documenta el diseño orientado a objetos, las decisiones tomadas, los resultados de las pruebas y el análisis de las tres políticas de asignación.

2. Objetivos

Objetivo general

Transformar los requerimientos de un simulador de Sistemas Operativos en un modelo de clases cohesivo, encapsulado y verificable, aplicando los principios de la programación orientada a objetos.

Objetivos específicos

- Modelar el ciclo de vida de un proceso con seis estados y transiciones que solo se puedan hacer por los caminos válidos.
- Implementar la asignación contigua de memoria con las políticas **First-Fit**, **Best-Fit** y **Worst-Fit**, intercambiables mediante polimorfismo.
- Liberar la memoria al terminar un proceso y fusionar automáticamente los bloques libres adyacentes.
- Planificar la CPU con **Round Robin** y quantum configurable, contando los cambios de contexto según la convención de la consigna.
- Simular bloqueos por Entrada/Salida de forma determinista.
- Calcular las métricas de ocupación de memoria, utilización de CPU, memoria libre, mayor bloque libre y **fragmentación externa**.
- Verificar cada requerimiento con pruebas automatizadas y una cobertura de líneas mayor al 90 %.
- Comparar el comportamiento de las tres políticas con la misma carga y analizar la fragmentación que producen.

3. Escenario y alcance

Un laboratorio universitario necesita un motor de simulación para estudiar cómo los procesos comparten una memoria limitada y una única CPU. La simulación arranca en el **tick 0**, con la memoria vacía (un solo bloque libre) y la CPU libre. La configuración de referencia es de **1024 KB de memoria** y un **quantum de 2 ticks**; los dos valores son configurables, igual que la política de asignación.

Alcance. La solución es una biblioteca: no tiene interfaz gráfica, menú de consola, función `main` ni script de demostración. Los métodos devuelven resultados y estados consultables, y la evidencia de funcionamiento son las pruebas. El avance del tiempo es determinista: no se usan esperas reales, temporizadores, hilos ni números aleatorios.

4. Descripción general de la solución

Aspecto	Decisión
Lenguaje	TypeScript en modo estricto (<code>strict</code> y <code>noImplicitOverride</code> , que obliga a marcar con <code>override</code> cada método redefinido).
Pruebas y cobertura	Vitest 5 con el proveedor <code>@vitest/coverage-v8</code> . El comando de cobertura falla si la cobertura de líneas no supera el 90 %.
Organización	<code>src/</code> (producción) separado de <code>tests/</code> (pruebas, con la misma estructura). Dentro de <code>src/</code> : <code>interfaces/</code> , <code>modelos/</code> y <code>politicas/</code> .
Versionado	Git y GitHub, con un commit por cada cambio chico y una rama por bloque de trabajo. Integración continua con GitHub Actions (<code>.github/workflows/ci.yml</code>): en cada push corre <code>npm ci</code> , <code>typecheck</code> y <code>test:coverage</code> .
Diagramas	draw.io: archivos editables <code>.drawio</code> y su versión en PNG en <code>docs/uml/</code> .

La idea central del diseño es que **el simulador coordina pero no hace el trabajo**. La memoria la administra `Memoria` , la CPU el `PlanificadorRoundRobin` , los bloqueos la `ColaBloqueados` , los eventos de E/S la `AgendaES` y las métricas el `RecolectorMetricas` . Cada `Proceso` cuida su propio estado. El simulador solo decide **el orden de las fases** de cada tick y traduce lo que pasó en la CPU en acciones.

5. Diseño: clases y responsabilidades

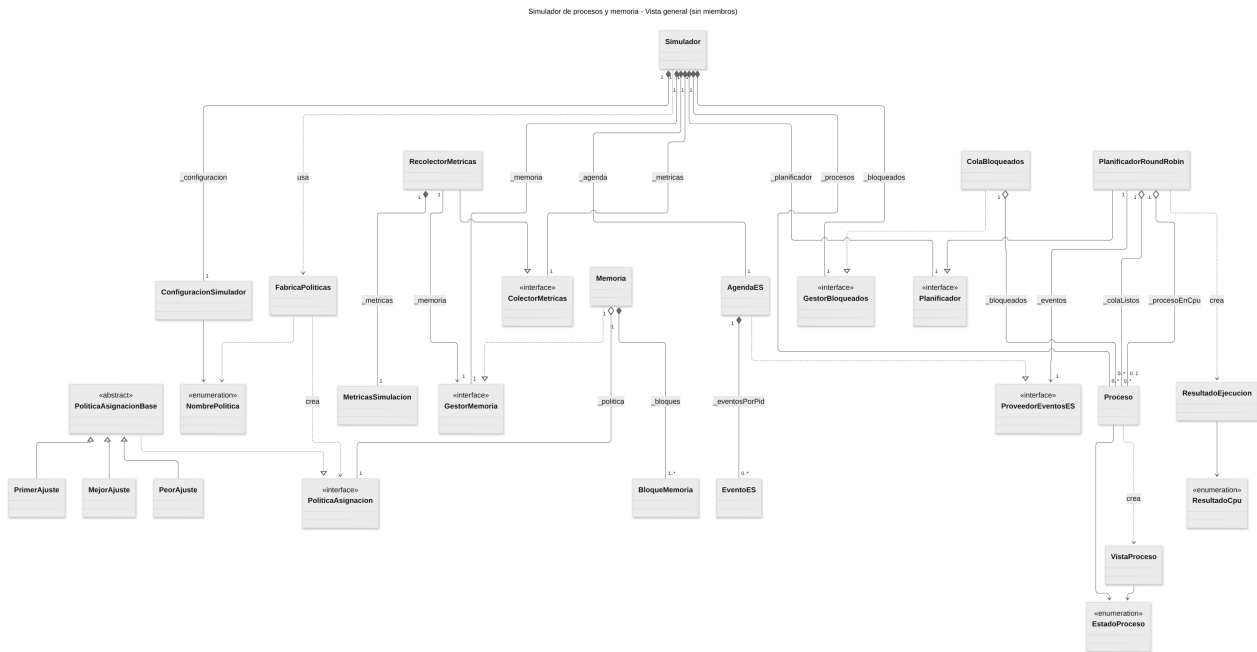


Figura 1. Vista general del diagrama de clases: clases, interfaces y relaciones (fuente editable: docs/uml/Diagrama de Clases.drawio).

Clase / interfaz	Tipo	Responsabilidad
Simulador	Clase	Coordina las cuatro fases de cada tick, registra procesos y expone el estado del sistema con vistas de solo lectura.
ConfiguracionSimulador	Clase	Valida memoria total, quantum y política antes de crear el simulador (RF01).
Proceso	Clase	Guarda PID, memoria, CPU total y restante, estado, quantum consumido y bloqueo restante. Solo cambia de estado por operaciones del dominio.
VistaProceso	Clase	Copia inmutable de un proceso: es lo único que se entrega hacia afuera.
EstadoProceso	Enumeración	Los seis estados del ciclo de vida.
GestorMemoria / Memoria	Interfaz / clase	Mapa ordenado de bloques: asigna, libera, fusiona y calcula las métricas de memoria.
BloqueMemoria	Clase	Tramo contiguo inmutable (inicio, tamaño y proceso o libre). Sabe dividirse y fusionarse.
PoliticaAsignacion	Interfaz	Contrato para elegir el bloque a usar en un pedido.
PoliticaAsignacionBase	Clase abstracta	Comparte el filtrado, el orden y el desempate por menor dirección.
PrimerAjuste, MejorAjuste, PeorAjuste	Clases	First-Fit, Best-Fit y Worst-Fit: cada una define solo su criterio de comparación.

Clase / interfaz	Tipo	Responsabilidad
FabricaPoliticass / NombrePolitica	Clase / enumeración	Crea la política elegida al configurar, con una tabla en lugar de condicionales.
Planificador / PlanificadorRoundRobin	Interfaz / clase	Cola FIFO de Listos y CPU: despacho, ejecución, quantum, finalización y bloqueo.
ResultadoEjecucion / ResultadoCpu	Clase / enumeración	Qué pasó en la CPU en un tick y a qué proceso.
ProveedorEventosES / AgendaES / EventoES	Interfaz / clases	Eventos deterministas de E/S: se validan al definirlos y el planificador los consulta.
GestorBloqueados / ColaBloqueados	Interfaz / clase	Procesos bloqueados y su temporizador.
ColectorMetricas / RecolectorMetricas / MetricasSimulacion	Interfaz / clases	Contadores del tick y métricas inmutables recalculadas al final de cada tick.

6. Decisiones de programación orientada a objetos

6.1 Doble encapsulamiento

La primera capa es que **ningún atributo es público**: todos son `private` y, cuando no deben cambiar, `readonly`. La segunda capa es que **no hay setters**: el estado solo cambia mediante operaciones del dominio que validan la regla correspondiente.

- `Proceso` concentra los cambios de estado en un único método privado, `transicionar(permitidos, destino)`, que lanza un error si la transición no es válida. Por ejemplo, `despachar()` solo funciona desde Listo y `terminar()` exige que la CPU restante sea cero.
- Hacia afuera nunca sale un `Proceso` real: el simulador, el planificador y la cola de bloqueados devuelven `VistaProceso`, una copia inmutable. Si se entregara el objeto real, se podría llamar a `despachar()` desde un test y saltar las reglas.
- Las colecciones se devuelven como **copias** (`[...this._bloques]`, `map(p => p.crearVista())`). Modificar la lista recibida no altera el simulador, y hay pruebas que lo verifican.
- `BloqueMemoria`, `EventoES`, `MetricasSimulacion` y `ResultadoEjecucion` son **inmutables**: cada operación devuelve un objeto nuevo, por eso se pueden exponer sin riesgo.

6.2 Herencia: dónde se usa y dónde no

Se usa herencia en un solo lugar: `PrimerAjuste`, `MejorAjuste` y `PeorAjuste` son una `PoliticaAsignacionBase`. La relación "es un" se cumple y se respeta la sustitución de Liskov: cualquiera de las tres sirve donde se espera una `PoliticaAsignacion`, y `polimorfismo.test.ts` lo comprueba corriendo los mismos casos sobre las tres.

En el resto del diseño se eligió **composición**. El `Simulador` no "es una" memoria ni "es un" planificador: los **tiene**. Tampoco se modelaron los estados del proceso como subclases, porque un proceso cambia de estado muchas veces durante su vida y un objeto no puede cambiar de clase. Por eso el estado es una enumeración y las transiciones se controlan con un método.

6.3 Clase abstracta

Las tres políticas hacen lo mismo salvo el criterio: filtran los bloques libres que alcanzan, los ordenan y desempatan por la menor dirección. `PoliticaAsignacionBase` implementa ese algoritmo común en `seleccionar()` y deja abstracto solo `comparar(a, b)`, que cada política define en una línea (patrón *Template Method*). Una interfaz sola no alcanzaba, porque no puede tener código; sin la clase abstracta, el filtrado y el desempate se repetirían tres veces.

6.4 Interfaces y principios SOLID

Hay seis interfaces: `GestorMemoria`, `PoliticaAsignacion`, `Planificador`, `GestorBloqueados`, `ColectorMetricas` y `ProveedorEventosES`. El simulador guarda sus partes tipadas por estas interfaces, no por las clases concretas.

Principio	Cómo se aplica
S — Responsabilidad única	Cada clase tiene un motivo de cambio: <code>Memoria</code> el mapa, el planificador la CPU, <code>RecolectorMetricas</code> las métricas. El <code>Simulador</code> solo coordina.
O — Abierto/cerrado	Para agregar una política nueva alcanza con una clase que extienda la base y una entrada en <code>FabricaPolíticas</code> ; <code>Memoria</code> no se toca.
L — Sustitución de Liskov	Las tres políticas son intercambiables y pasan las mismas pruebas.
I — Segregación de interfaces	El planificador solo necesita saber si hay un evento de E/S, por eso depende de <code>ProveedorEventosES</code> , que tiene un único método, y no de toda la <code>AgendaES</code> .
D — Inversión de dependencias	<code>Memoria</code> recibe una <code>PoliticaAsignacion</code> por constructor; <code>RecolectorMetricas</code> depende de <code>GestorMemoria</code> . Eso permite probarlas con dobles.

6.5 Polimorfismo sin condicionales por tipo

- `Memoria.asignar()` llama a `this._politica.seleccionar(...)` sin saber qué política tiene: el comportamiento cambia según el objeto, no según un `if`.
- `FabricaPolíticas` usa una tabla `Record<NombrePolitica, () => PoliticaAsignacion>` en lugar de un `switch`.
- El `Simulador` traduce el resultado de la CPU en acciones con otra tabla, `Record<ResultadoCpu, (pid) => void>`: por ejemplo, `FINALIZO` libera la memoria y `BLOQUEO` agrega el proceso a la cola de bloqueados.
- En las pruebas, `it.each` corre el mismo escenario con las tres políticas y solo cambia el resultado esperado.

6.6 Composición y colaboración

El `Simulador` crea en su constructor la `Memoria` (con la política que entrega la fábrica), la `AgendaES`, el `PlanificadorRoundRobin`, la `ColaBloqueados` y el `RecolectorMetricas`: son sus partes y viven mientras él viva (composición). `Memoria` compone sus `BloqueMemoria`. El planificador y la cola de bloqueados, en cambio, guardan **referencias** a procesos que pertenecen al simulador; por eso en el diagrama esa relación es una agregación.

6.7 Colecciones de objetos

Cada colección está encapsulada dentro de la clase que la administra y se expone solo como copia o vista: el `Map<number, Proceso>` de procesos del simulador, el arreglo de bloques de la memoria, la cola FIFO del planificador, la lista de bloqueados y el `Map<number, EventoES[]>` de la agenda. Las multiplicidades del diagrama de clases (0..*, 1..*, 0..1) corresponden a estas colecciones.

7. Estructuras de datos utilizadas

Estructura	Dónde	Por qué
<code>Map<number, Proceso></code>	<code>Simulador._procesos</code>	Búsqueda por PID en tiempo constante y conserva el orden de registro, que es el orden en que se reintenta la admisión (RF03).
<code>BloqueMemoria[]</code> ordenado por dirección	<code>Memoria._bloques</code>	Representa la memoria contigua: los vecinos de un bloque son las posiciones $i - 1$ e $i + 1$, lo que hace directa la coalescencia.
<code>Proceso[]</code> como cola FIFO	<code>PlanificadorRoundRobin._colaListos</code>	<code>push</code> agrega al final y <code>shift</code> despacha al primero, como pide Round Robin.
<code>Proceso null</code>	<code>PlanificadorRoundRobin._procesoEnCpu</code>	Hay una sola CPU: a lo sumo un proceso ejecutando.
<code>Proceso[]</code>	<code>ColaBloqueados._bloqueados</code>	Mantiene el orden de bloqueo, que define el orden en que vuelven a Listos.
<code>Map<number, EventoES[]></code>	<code>AgendaES._eventosPorPid</code>	Un proceso puede tener varios eventos de E/S en puntos distintos.
<code>number[]</code>	<code>Simulador._terminados</code>	Orden en que terminaron los procesos.
<code>ReadOnlySet<ResultadoCpu></code>	<code>RecolectorMetricas</code>	Conjunto de resultados que cuentan como cambio de contexto.

8. Estados de los procesos y diagrama de transición

Estado	Significado
Nuevo	Recién registrado; todavía no se intentó asignarle memoria.
Esperando Memoria	No había un bloque libre suficiente; se reintenta al inicio de cada tick.
Listo	Tiene memoria asignada y espera en la cola FIFO.
Ejecutando	Está usando la única CPU.
Bloqueado	Está en una operación de E/S: no usa CPU pero conserva su memoria.
Terminado	Agotó su CPU; liberó la memoria y no vuelve a ninguna cola.

Diagrama de estados del Proceso (cada transición es un método de Proceso)

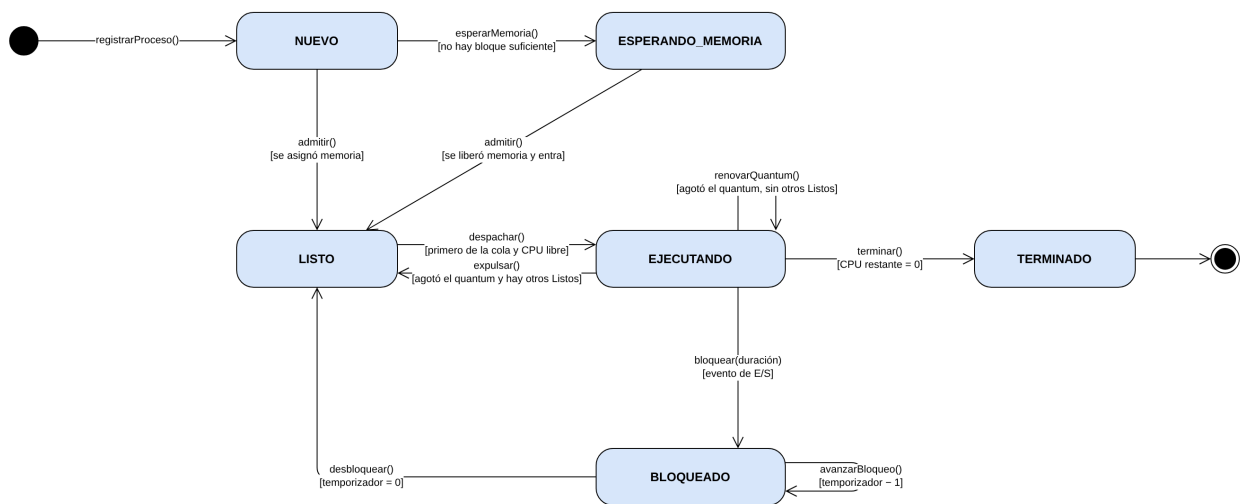


Figura 3. Diagrama de transición de estados del proceso. Cada flecha es un método de Proceso y entre corchetes va la condición.

Cualquier transición que no esté en el diagrama lanza un error. Por ejemplo, no se puede despachar un proceso Bloqueado ni volver a admitir uno Terminado; las pruebas de `Proceso.test.ts` verifican cada camino válido y varios inválidos.

9. Orden de cada tick y planificación Round Robin

Cada llamada a `avanzarTick()` avanza **exactamente una unidad** y ejecuta siempre las mismas cuatro fases, en este orden:

1. **Admisión:** se recorre a los procesos Nuevos y Esperando Memoria en orden de registro y se intenta asignarles memoria. Si uno no cabe, se sigue con el próximo: no frena a los que sí caben.
2. **Bloqueados:** a cada proceso bloqueado se le resta un tick de temporizador; los que llegan a cero vuelven al final de la cola de Listos y pueden ejecutarse en ese mismo tick.
3. **CPU:** si la CPU está libre se despacha al primero de la cola (quantum en cero), y el proceso en CPU consume una unidad.
4. **Reloj y métricas:** el tick aumenta en uno y se recalculan las métricas.

Al final de la fase de CPU se resuelve qué pasa con el proceso, con esta prioridad: **finalización > bloqueo por E/S > agotamiento del quantum**. Si se agotó el quantum y hay otros Listos, el proceso vuelve al final de la cola (expulsión); si no hay otros, renueva el quantum y sigue sin cambio de contexto. La memoria que se libera al terminar un proceso recién se ofrece en la admisión del tick siguiente.

Cambios de contexto (convención de RF09). Se cuenta uno por cada expulsión por quantum con otros Listos y por cada bloqueo por E/S. No cuentan el despacho inicial, la finalización ni la renovación del quantum.

Ejemplo: caso 4 del Anexo I (Q = 2, P1 con CPU 3, P2 con CPU 2)

Tick	En CPU	CPU restante P1 / P2	Qué pasa	Cambios de contexto
1	P1	2 / 2	Se admiten los dos; P1 se despacha y ejecuta.	0
2	P1	1 / 2	P1 agota el quantum y hay otro Listo: vuelve al final de la cola.	1
3	P2	1 / 1	P2 se despacha y ejecuta.	1
4	P2	1 / 0	P2 termina justo en el límite del quantum: no se reencola y libera su memoria.	1
5	P1	0 / 0	P1 vuelve a la CPU y termina.	1

El orden de ejecución es P1, P1, P2, P2, P1 con un único cambio de contexto, que es exactamente lo que verifica la prueba del caso 4.

10. Gestión de memoria: políticas de asignación y coalescencia

Representación

La memoria es una lista de bloques ordenada por dirección. Cada bloque tiene inicio, tamaño y el PID del proceso o `null` si está libre. La clase `Memoria` garantiza siempre cuatro invariantes: los bloques cubren toda la memoria sin huecos, no se solapan, la suma de tamaños es igual a la memoria total y nunca quedan dos bloques libres juntos. Las pruebas lo comprueban después de cada operación con un verificador de invariantes.

Asignación

La política elige un bloque libre suficiente. Si el bloque es más grande que el pedido, se divide en una parte ocupada y un resto libre; si el ajuste es exacto, se ocupa entero y no se crea un bloque de tamaño cero. Si ningún bloque alcanza, `asignar()` devuelve `false` **sin modificar el mapa**, aunque la suma de la memoria libre alcance.

Política	Clase	Criterio	Empate
First-Fit	<code>PrimerAjuste</code>	El primer bloque suficiente por dirección.	Menor dirección
Best-Fit	<code>MejorAjuste</code>	El bloque suficiente de menor tamaño (deja el hueco más chico).	Menor dirección
Worst-Fit	<code>PeorAjuste</code>	El bloque suficiente de mayor tamaño (deja el hueco más grande).	Menor dirección

La política se elige al configurar el simulador (`new ConfiguracionSimulador(1024, 2, NombrePolitica.BEST_FIT)`); si no se indica, se usa First-Fit.

Liberación y coalescencia (pseudocódigo)

```

liberar(pid):
  i ← posición del bloque cuyo proceso es pid
  si no existe: error "el proceso no tiene memoria asignada"
  bloques[i] ← bloques[i] liberado // pasa a estar libre

  // 1) vecino derecho
  si existe bloques[i + 1] y está libre:
    reemplazar bloques[i] y bloques[i + 1] por su fusión

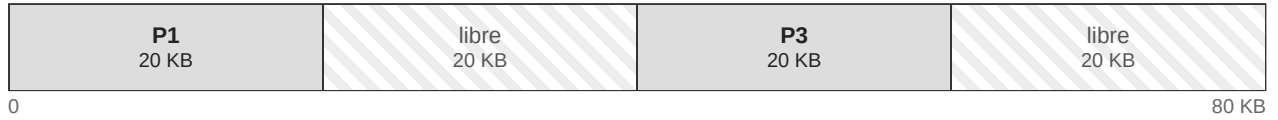
  // 2) vecino izquierdo
  si existe bloques[i - 1] y está libre:
    reemplazar bloques[i - 1] y bloques[i] por su fusión

```

Primero se fusiona a la derecha porque eso no cambia la posición *i*; después, la fusión a la izquierda absorbe el bloque ya agrandado. Así se cubren los tres casos (vecino izquierdo, derecho o ambos)

con una sola pasada. La coalescencia **no mueve bloques ocupados**: no es compactación. Cuando terminan todos los procesos, la memoria vuelve a ser un único bloque libre del tamaño total.

Antes de liberar P3: libres a ambos lados



Después: un solo bloque libre de 60 KB (caso «ambos vecinos» de las pruebas)



11. Diagramas de secuencia

Se presentan tres diagramas de secuencia asociados a requerimientos distintos. Los objetos, los mensajes y los fragmentos `loop`, `opt` y `alt` corresponden a los métodos reales del código.

- **Secuencia 1 (RF06):** el simulador no ejecuta la lógica de cada fase: le pide a la cola de bloqueados que avance, encola a los que terminaron su E/S, le pide al planificador que ejecute un tick, reacciona al resultado y le avisa al recolector de métricas.
- **Secuencia 2 (RF02, RF03 y RF04):** la memoria le delega la elección del bloque a la política mediante `seleccionar()`, sin saber cuál es (polimorfismo), y le pasa una copia del mapa. El fragmento `alt` muestra los dos resultados posibles: el proceso pasa a Listo o queda Esperando Memoria.
- **Secuencia 3 (RF07, RF08 y RF05):** el fragmento `alt` sigue la prioridad del enunciado: primero se revisa si el proceso terminó (y entonces se libera su memoria con coalescencia), después si hay un evento de E/S, y por último el quantum.

Los archivos editables están en `docs/uml/` con el nombre `Diagrama de Secuencia-N-....drawio`.

Secuencia 1 - RF06: avanzar un tick (cuatro fases)

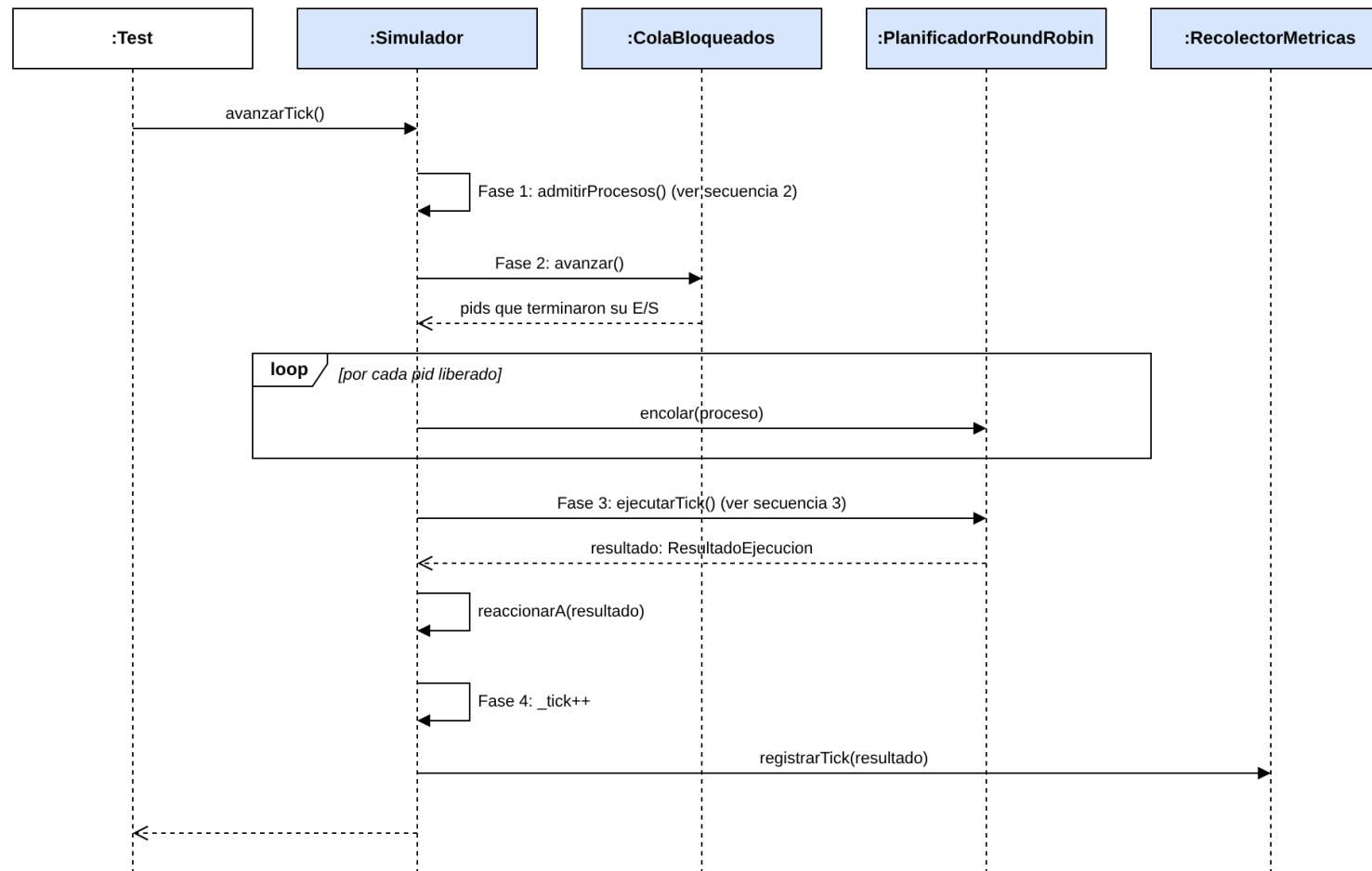


Figura 4. Secuencia 1 — RF06: avanzar un tick con sus cuatro fases.

Secuencia 2 - RF02, RF03 y RF04: registro, admisión y asignación contigua

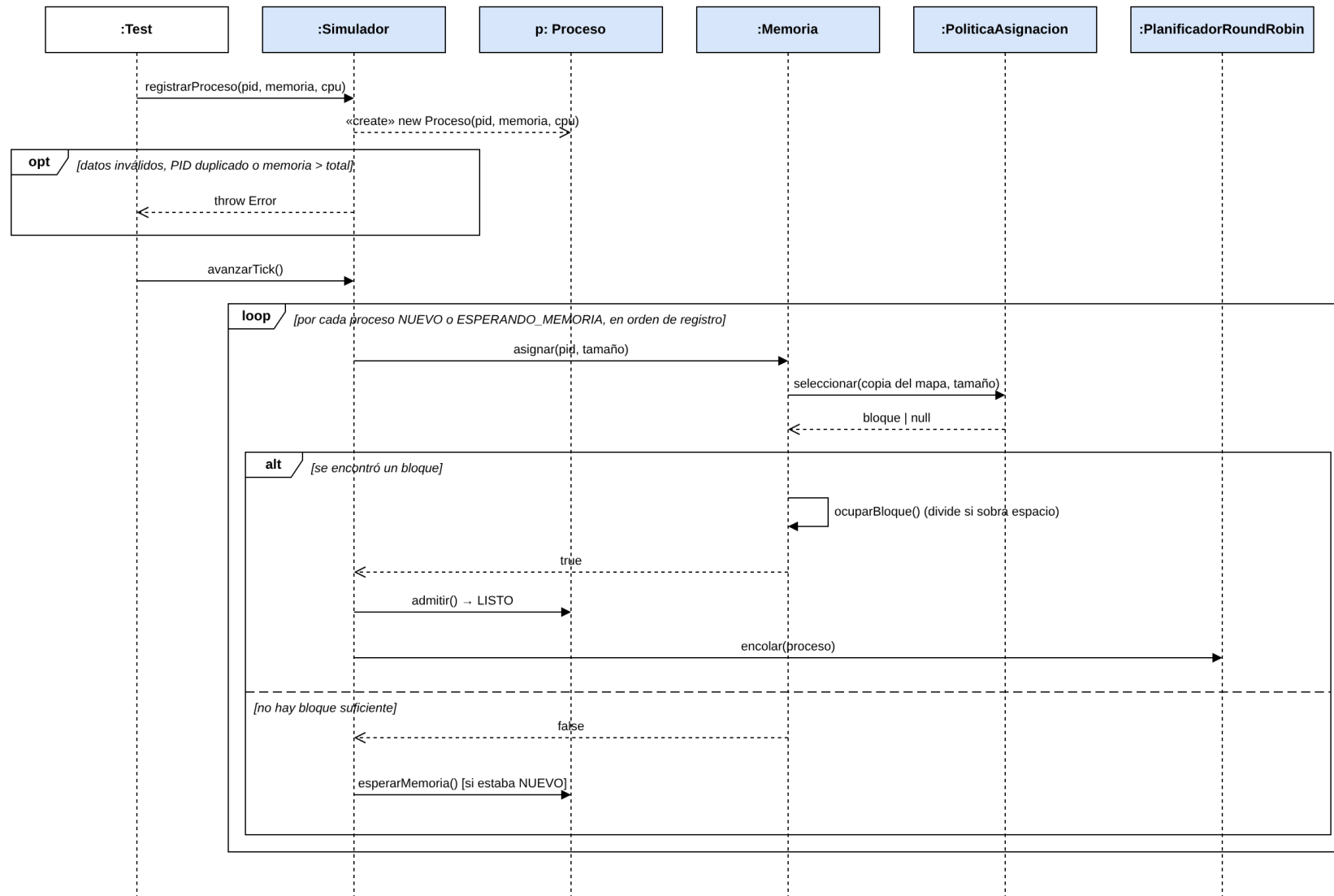


Figura 5. Secuencia 2 — RF02, RF03 y RF04: registro, admisión y asignación contigua.

Secuencia 3 - RF07, RF08 y RF05: ejecución Round Robin, bloqueo por E/S y liberación con coalescencia

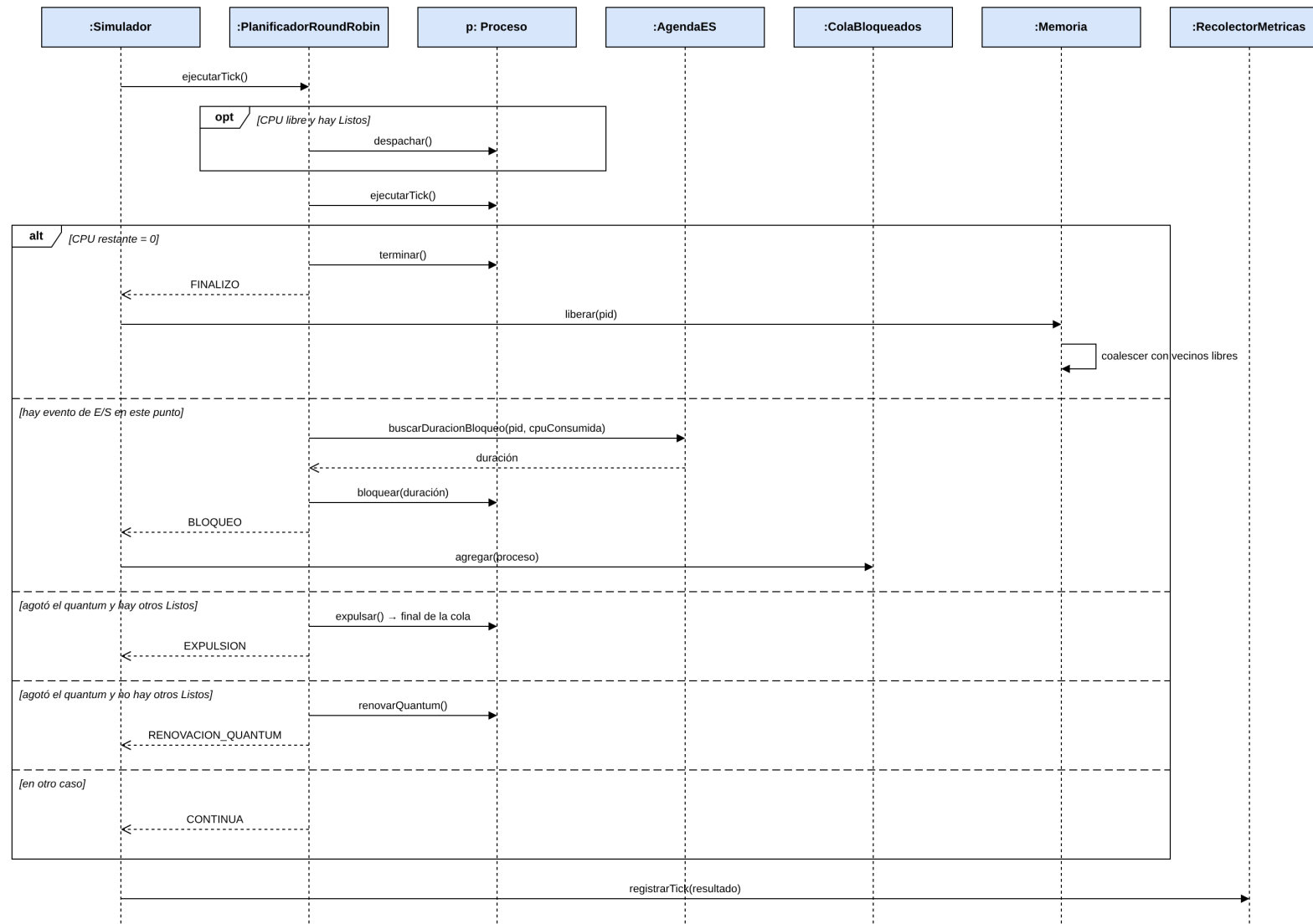


Figura 6. Secuencia 3 — RF07, RF08 y RF05: ejecución Round Robin, bloqueo por E/S y liberación con coalescencia.

12. Matriz de trazabilidad RF → clase → pruebas

RF	Clase / método	Pruebas
RF01 Configurar e iniciar	<code>ConfiguracionSimulador</code> , constructor de <code>Simulador</code> y <code>Memoria</code>	<code>ConfiguracionSimulador.test.ts</code> , <code>Simulador.test.ts</code> (configuración e inicio), Anexo I caso 1
RF02 Registrar y consultar	<code>Simulador.registrarProceso</code> , <code>obtenerProceso</code> , <code>obtenerProcesos</code> , <code>Proceso</code> , <code>VistaProceso</code>	<code>Proceso.test.ts</code> (creación), <code>Simulador.test.ts</code> (registro y consulta), caso 1
RF03 Estados y admisión	<code>Proceso.transicionar</code> , <code>Simulador.admitirProcesos</code>	<code>Proceso.test.ts</code> (admisión), <code>Simulador.test.ts</code> (admisión), casos 2 y 8
RF04 Memoria contigua	<code>Memoria.asignar</code> , <code>BloqueMemoria.separar</code> , las tres políticas, <code>FabricaPoliticas</code>	<code>Memoria.test.ts</code> , <code>Memoria.politicas.test.ts</code> , pruebas de cada política, <code>polimorfismo.test.ts</code> , caso 2, comparación de políticas
RF05 Liberación y coalescencia	<code>Memoria.liberar</code> , <code>BloqueMemoria.fusionarCon</code>	<code>Memoria.test.ts</code> (liberar y coalescencia), <code>BloqueMemoria.test.ts</code> , caso 3
RF06 Tick determinista	<code>Simulador.avanzarTick</code> (cuatro fases)	<code>Simulador.test.ts</code> (avance del reloj y orden de fases), caso 8
RF07 Round Robin	<code>PlanificadorRoundRobin</code>	<code>PlanificadorRoundRobin.test.ts</code> , casos 4 y 5
RF08 Entrada/Salida	<code>EventoES</code> , <code>AgendaES</code> , <code>ColaBloqueados</code> , <code>Simulador.definirEventoES</code>	<code>EventoES.test.ts</code> , <code>AgendaES.test.ts</code> , <code>ColaBloqueados.test.ts</code> , <code>Simulador.test.ts</code> (E/S), caso 6
RF09 Métricas	<code>RecolectorMetricas</code> , <code>MetricasSimulacion</code> , métricas de <code>Memoria</code>	<code>RecolectorMetricas.test.ts</code> , <code>Memoria.test.ts</code> (métricas), caso 7
RF10 Estado del sistema	Consultas del <code>Simulador</code> (vistas y copias)	<code>Simulador.invariantes.test.ts</code> , <code>Proceso.test.ts</code> (vista), caso 8

13. Casos de prueba y resultados

Hay **316 pruebas en 21 archivos**: pruebas unitarias por clase (casos normales, límites y errores), pruebas de colaboración del simulador que verifican el orden de las fases, un archivo con los 8 casos mínimos del Anexo I y uno con la comparación de políticas. Todas son independientes y repetibles, y todas tienen aserciones sobre resultados, estados, colas o métricas.

Casos mínimos de verificación (Anexo I)

Caso	Datos	Resultado verificado	Estado
1. Configuración y registro	1024 KB y $Q = 2$; memoria 0 o -10, quantum 0 o 1,5; PID repetido; proceso de 101 KB con 100 KB	Tick 0, un único bloque libre, CPU libre y colas vacías; cada dato inválido se rechaza sin dejar estados parciales.	✓
2. Asignación y espera	Proceso de 30 KB; de 100 KB en 100 KB; huecos de 30, 20 y 40 KB y un pedido de 15 KB	Partición parcial, ajuste exacto, First-Fit en 0, Best-Fit en 40 y Worst-Fit en 70; un fracaso no modifica el mapa y el proceso entra al liberarse memoria.	✓
3. Coalescencia	Bloques de 20 KB con quantum 1	Fusión con el vecino izquierdo, con el derecho y con ambos; al terminar todos queda un bloque libre del tamaño total.	✓
4. Round Robin	$Q = 2$, P1 con CPU 3 y P2 con CPU 2	Ejecución P1, P1, P2, P2, P1 y un cambio de contexto.	✓
5. Quantum y finalización	Un único proceso; P1 que termina en el límite del quantum	Renovación sin cambio de contexto; el que termina no se reencola.	✓
6. Bloqueo por E/S	P1 con evento después de 1 tick de CPU y 2 de duración	Se bloquea, conserva la memoria, no consume CPU, vuelve a Listos al vencer el temporizador y suma un cambio de contexto.	✓
7. Métricas y límites	Huecos no contiguos de 100 y 300 KB en 500 KB; memoria llena; tick 0	Libre total 400, mayor hueco 300 y fragmentación 25 %; memoria llena con 100 % de ocupación y 0 % de fragmentación; tick 0 con 0 % de CPU.	✓
8. Orden e invariantes	Liberación al final de un tick; 4 procesos con eventos de E/S durante 20 ticks	La admisión ocurre recién en el tick siguiente; en cada tick no hay duplicados, solapamientos ni dos procesos en CPU.	✓

Capturas de la ejecución

```

PS C:\paradigmas\AE2\simulador-de-memoria> npx vitest run tests/modelos/Simulador.anexoI.test.ts --reporter=verbose

RUN v5.0.3 C:\paradigmas\AE2\simulador-de-memoria
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > el estado inicial es correcto 5ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza parámetros inválidos (memoria 0, quantum 2) 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza parámetros inválidos (memoria -10, quantum 2) 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza parámetros inválidos (memoria 100, quantum 0) 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza parámetros inválidos (memoria 100, quantum 1.5) 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza un PID repetido 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza un proceso mayor que la memoria 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > partición parcial: el proceso ocupa el inicio y el resto queda libre 3ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > ajuste exacto: el proceso ocupa todo el bloque 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > selección según política: FIRST_FIT ubica el proceso en 0 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > selección según política: BEST_FIT ubica el proceso en 40 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > selección según política: WORST_FIT ubica el proceso en 70 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > si no hay bloque suficiente falla sin modificar el mapa y lo admite al liberarse memoria 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 3: coalescencia > fusiona con el vecino izquierdo 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 3: coalescencia > fusiona con el vecino derecho 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 3: coalescencia > fusiona con ambos vecinos 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 3: coalescencia > al liberar todos los procesos queda un único bloque libre del tamaño total 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 4: Round Robin > Q = 2, P1 con CPU 3 y P2 con CPU 2: ejecutan P1, P1, P2, P2, P1 con un cambio de contexto 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 5: quantum y finalización > un único proceso renueva su quantum sin cambio de contexto 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 5: quantum y finalización > finalizar en el límite del quantum no lo reencola 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 6: bloqueo por E/S > bloquea, conserva la memoria, no consume CPU y retorna a Listos al vencer el temporizador 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 7: métricas y límites > huecos no contiguos de 100 y 300 KB: libre total 400, mayor hueco 300 y fragmentación 25% 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 7: métricas y límites > con la memoria llena: ocupación 100%, sin memoria libre y fragmentación 0% 2ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 7: métricas y límites > en el tick 0 la utilización de CPU es 0% y toda la memoria está libre 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 8: orden e invariantes > una liberación al final del tick habilita la admisión recién en el siguiente 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 8: orden e invariantes > en cada tick no hay duplicados, solapamientos ni dos procesos en CPU 4ms

Test Files 1 passed (1)
Tests 26 passed (26)
Start at 21:43:50
Duration 467ms (transform 71%, import 17%, tests 9%, worker 3%)

PS C:\paradigmas\AE2\simulador-de-memoria>

```

Figura 7. Ejecución de los 8 casos mínimos del Anexo I (`npx vitest run tests/modelos/Simulador.anexoI.test.ts --reporter=verbose`): 26 pruebas aprobadas.

```

PS C:\paradigmas\AE2\simulador-de-memoria> npx vitest run tests/modelos/Simulador --reporter=verbose

✓ tests/modelos/Simulador.test.ts > Simulador - validación de eventos de E/S (RF08) > rechaza datos inválidos (1, 0) 0ms
✓ tests/modelos/Simulador.test.ts > Simulador - validación de eventos de E/S (RF08) > rechaza datos inválidos (1, -2) 0ms
✓ tests/modelos/Simulador.test.ts > Simulador - validación de eventos de E/S (RF08) > rechaza datos inválidos (1, 1.5) 0ms
✓ tests/modelos/Simulador.test.ts > Simulador - validación de eventos de E/S (RF08) > rechaza un punto que nunca se alcanzaría (5 con CPU total 5) 0ms
✓ tests/modelos/Simulador.test.ts > Simulador - validación de eventos de E/S (RF08) > rechaza un punto que nunca se alcanzaría (6 con CPU total 5) 1ms
✓ tests/modelos/Simulador.test.ts > Simulador - validación de eventos de E/S (RF08) > rechaza un punto que ya pasó 1ms
✓ tests/modelos/Simulador.test.ts > Simulador - validación de eventos de E/S (RF08) > rechaza dos eventos en el mismo punto del mismo proceso 1ms
✓ tests/modelos/Simulador.test.ts > Simulador - bloqueo por E/S (RF06, RF08, RF09) > bloquea al proceso, conserva su memoria y deja de consumir CPU mientras espera 3ms
✓ tests/modelos/Simulador.test.ts > Simulador - bloqueo por E/S (RF06, RF08, RF09) > al vencer el temporizador retorna a Listo y se despacha en ese mismo tick 1ms
✓ tests/modelos/Simulador.test.ts > Simulador - bloqueo por E/S (RF06, RF08, RF09) > el proceso que vuelve del bloqueo se encola al final de la cola de Listos 1ms
✓ tests/modelos/Simulador.test.ts > Simulador - bloqueo por E/S (RF06, RF08, RF09) > el bloqueo tiene prioridad sobre la rotación por quantum 1ms
✓ tests/modelos/Simulador.test.ts > Simulador - bloqueo por E/S (RF06, RF08, RF09) > tras bloquearse y volver, el proceso termina y libera su memoria 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > el estado inicial es correcto 9ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza parámetros inválidos (memoria 0, quantum 2) 2ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza parámetros inválidos (memoria -10, quantum 2) 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza parámetros inválidos (memoria 100, quantum 0) 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza parámetros inválidos (memoria 100, quantum 1.5) 0ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza un PID repetido 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 1: configuración y registro > rechaza un proceso mayor que la memoria 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > partición parcial: el proceso ocupa el inicio y el resto queda libre 2ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > ajuste exacto: el proceso ocupa todo el bloque 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > selección según política: FIRST_FIT ubica el proceso en 0 5ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > selección según política: BEST_FIT ubica el proceso en 40 3ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > selección según política: WORST_FIT ubica el proceso en 70 2ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 2: asignación y espera > si no hay bloque suficiente falla sin modificar el mapa y lo admite al liberarse memoria 4ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 3: coalescencia > fusiona con el vecino izquierdo 2ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 3: coalescencia > fusiona con el vecino derecho 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 3: coalescencia > fusiona con ambos vecinos 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 3: coalescencia > al liberar todos los procesos queda un único bloque libre del tamaño total 8ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 4: Round Robin > Q = 2, P1 con CPU 3 y P2 con CPU 2: ejecutan P1, P1, P2, P2, P1 con un cambio de contexto 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 5: quantum y finalización > un único proceso renueva su quantum sin cambio de contexto 2ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 5: quantum y finalización > finalizar en el límite del quantum no lo reencola 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 6: bloqueo por E/S > bloquea, conserva la memoria, no consume CPU y retorna a Listos al vencer el temporizador 9ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 7: métricas y límites > huecos no contiguos de 100 y 300 KB: libre total 400, mayor hueco 300 y fragmentación 25% 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 7: métricas y límites > con la memoria llena: ocupación 100%, sin memoria libre y fragmentación 0% 6ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 7: métricas y límites > en el tick 0 la utilización de CPU es 0% y toda la memoria está libre 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 8: orden e invariantes > una liberación al final del tick habilita la admisión recién en el siguiente 1ms
✓ tests/modelos/Simulador.anexoI.test.ts > Anexo I - Caso 8: orden e invariantes > en cada tick no hay duplicados, solapamientos ni dos procesos en CPU 21ms

Test Files 4 passed (4)
Tests 98 passed (98)
Start at 21:47:28
Duration 932ms (transform 56%, import 29%, tests 12%, worker 4%)

PS C:\paradigmas\AE2\simulador-de-memoria>

```

Figura 8. Ejecución de todas las pruebas de colaboración del Simulador (`npx vitest run tests/modelos/Simulador --reporter=verbose`): 4 archivos y 98 pruebas aprobadas.

14. Cobertura de código

Ítem	Detalle
Herramienta	Vitest 5 con el proveedor <code>@vitest/coverage-v8</code>
Comando reproducible	<code>npm run test:coverage</code>
Alcance medido	Todos los archivos de <code>src/</code> (<code>include: ["src/**/*.ts"]</code>), aunque ningún test los importe. Se excluye <code>src/interfaces/</code> , que solo declara tipos y no genera código ejecutable.
Umbral	El comando falla si la cobertura de líneas no supera el 90 %.
Resultado del commit entregado	100 % de líneas (359/359), sentencias (375/375), funciones (174/174) y ramas (126/126), con 316 pruebas aprobadas .

Captura: reporte en la terminal

```

PS C:\paradigmas\AE2\simulador-de-memoria> npm run test:coverage
✓ tests/políticas/PolíticaAsignacionBase.test.ts (6 tests) 15ms
✓ tests/modelos/EventoES.test.ts (9 tests) 19ms
✓ tests/modelos/ResultadoEjecucion.test.ts (2 tests) 17ms
✓ tests/políticas/MejorAjuste.test.ts (6 tests) 11ms
✓ tests/políticas/PeorAjuste.test.ts (5 tests) 11ms
✓ tests/políticas/PrimerAjuste.test.ts (5 tests) 10ms

Test Files 21 passed (21)
Tests 316 passed (316)
Start at 21:48:03
Duration 3.21s (transform 41%, import 29%, worker 21%, tests 8%)

Transform transforming modules took 3.19s - 41% of tracked time, re-done on every run
persist transforms across runs with fsModuleCache: true
learn more: https://vitest.dev/guide/ improving-performance# caching-between-reruns

% Coverage report from v8
-----
File                               % Stats % Branch % Funcs % Lines Uncovered Line #s
-----
All Files                          100      100      100      100
modelos                             100      100      100      100
  AgendaES.ts                      100      100      100      100
  BloqueMemoria.ts                 100      100      100      100
  Colaboradores.ts                100      100      100      100
  ConfiguracionSimulador.ts        100      100      100      100
  EstadoProceso.ts                 100      100      100      100
  EventoES.ts                      100      100      100      100
  Memoria.ts                      100      100      100      100
  MetricasSimulacion.ts            100      100      100      100
  PlanificadorRoundRobin.ts        100      100      100      100
  Proceso.ts                      100      100      100      100
  RecolectorMetricas.ts            100      100      100      100
  ResultadoEjecucion.ts            100      100      100      100
  Simulador.ts                    100      100      100      100
  VistaProceso.ts                 100      100      100      100
politicas                          100      100      100      100
  FabricaPoliticas.ts              100      100      100      100
  MejorAjuste.ts                   100      100      100      100
  NombrePolitica.ts               100      100      100      100
  PeorAjuste.ts                   100      100      100      100
  PoliticaAsignacionBase.ts        100      100      100      100
  PrimerAjuste.ts                  100      100      100      100
-----
  
```

Figura 9. `npm run test:coverage`: 21 archivos y 316 pruebas aprobadas, con 100 % en todas las columnas.

Captura: reporte HTML (coverage/index.html)

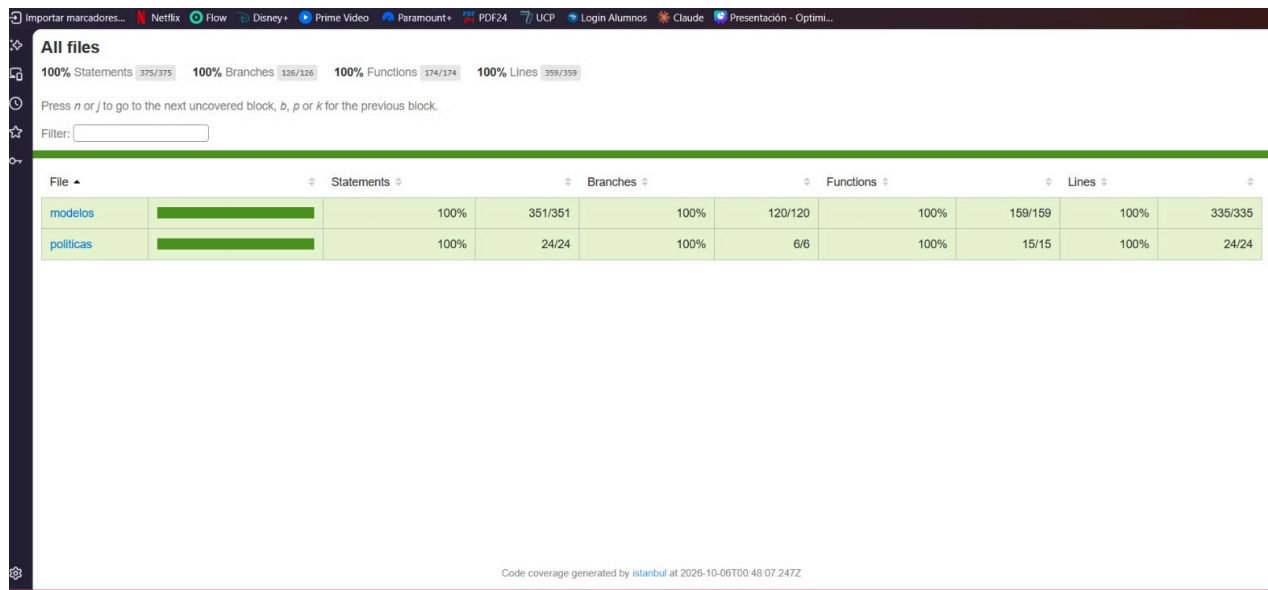


Figura 10. Reporte HTML de cobertura (coverage/index.html): 375/375 sentencias, 126/126 ramas, 174/174 funciones y 359/359 líneas.

15. Análisis comparativo First-Fit, Best-Fit y Worst-Fit

15.1 Metodología

El archivo `tests/modelos/Simulador.comparacionPoliticas.test.ts` corre **la misma carga con las tres políticas** y solo cambia la política elegida al configurar. En una memoria de 100 KB con quantum 1, cuatro procesos dejan dos huecos no contiguos separados por un proceso que sigue vivo; después se registran los mismos pedidos y se avanza un tick para que se intente admitirlos. Para cada política se verifica el mapa final, qué procesos quedaron esperando, la memoria libre, el mayor hueco y la fragmentación externa.

Punto de partida de los escenarios A y B (igual para las tres políticas): huecos de 30 KB y 20 KB, fragmentación 40 %



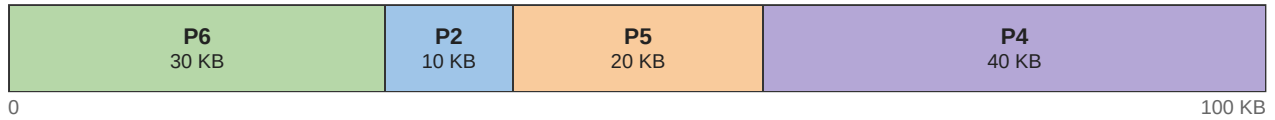
15.2 Resultados

Escenario	Política	Mapa resultante	En espera	Libre / mayor hueco	Fragmentación
A. Huecos 30 y 20, pedidos de 20 y 30 KB	First-Fit	P5 en 0 (sobran 10 KB); P6 no entra	P6	30 / 20 KB	33,3 %
	Best-Fit	P5 ocupa el hueco de 20 y P6 el de 30, exactos	—	0 / 0 KB	0 %
	Worst-Fit	P5 en el hueco de 30 (sobran 10 KB); P6 no entra	P6	30 / 20 KB	33,3 %
B. Huecos 30 y 20, pedidos de 15, 15 y 20 KB	First-Fit	Los dos de 15 llenan el hueco de 30; el de 20 ocupa el de 20	—	0 / 0 KB	0 %
	Best-Fit	El primer 15 rompe el hueco de 20 (sobran 5)	P7	20 / 15 KB	25 %
	Worst-Fit	El segundo 15 va al hueco de 20 (sobran 5)	P7	20 / 15 KB	25 %
C. Huecos 20 y 30, pedidos de 5, 20 y 20 KB	First-Fit	El de 5 rompe el hueco de 20; queda un hueco de 15	P7	25 / 15 KB	40 %
	Best-Fit	Igual que First-Fit: el hueco de 20 es el más justo	P7	25 / 15 KB	40 %
	Worst-Fit	El de 5 va al hueco de 30; el hueco de 20 queda entero para el último	—	5 / 5 KB	0 %

Escenario A con First-Fit y Worst-Fit: P6 (30 KB) espera aunque hay 30 KB libres



Escenario A con Best-Fit: los dos pedidos entran en ajuste exacto



15.3 Interpretación

- **Ninguna política es mejor en todos los casos.** Con la misma memoria de partida, cada una es la única que admite a todos en algún escenario.
- **Best-Fit** rinde cuando los pedidos coinciden con los huecos: usa cada uno de forma exacta. Pero cuando no coinciden deja **restos chicos** (los 5 KB del escenario B) que después no le sirven a nadie.
- **Worst-Fit** parte el hueco más grande y deja restos grandes que todavía sirven; además conserva enteros los huecos chicos para pedidos que los usan justo (escenario C). Su costo es que consume rápido los bloques grandes, y un pedido grande posterior puede no encontrar lugar.
- **First-Fit** es el más simple: no recorre todos los bloques para comparar tamaños y en muchas cargas da resultados iguales o mejores que las otras dos (escenario B). Por eso es la política por defecto del simulador.

Captura: ejecución de la comparación

```

PS C:\paradigmas\AE2\simulador-de-memoria> npx vitest run tests/modelos/Simulador.comparacionPolíticas.test.ts --reporter=verbose

 RUN  v5.0.3 C:\paradigmas\AE2\simulador-de-memoria

✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > el punto de partida es el mismo para FIRST_FIT: huecos de 30 KB en 0 y de 20 KB en 40
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > el punto de partida es el mismo para BEST_FIT: huecos de 30 KB en 0 y de 20 KB en 40
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > el punto de partida es el mismo para WORST_FIT: huecos de 30 KB en 0 y de 20 KB en 40
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > pedidos de 20 y 30 KB con FIRST_FIT: solo Best-Fit admite a ambos 1ms
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > pedidos de 20 y 30 KB con BEST_FIT: solo Best-Fit admite a ambos 0ms
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > pedidos de 20 y 30 KB con WORST_FIT: solo Best-Fit admite a ambos 1ms
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > pedidos de 15, 15 y 20 KB con FIRST_FIT: solo First-Fit admite a todos 1ms
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > pedidos de 15, 15 y 20 KB con BEST_FIT: solo First-Fit admite a todos 0ms
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > pedidos de 15, 15 y 20 KB con WORST_FIT: solo First-Fit admite a todos 0ms
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > pedidos de 5, 20 y 20 KB con FIRST_FIT: solo Worst-Fit admite a todos 1ms
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > pedidos de 5, 20 y 20 KB con BEST_FIT: solo Worst-Fit admite a todos 0ms
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > pedidos de 5, 20 y 20 KB con WORST_FIT: solo Worst-Fit admite a todos 0ms
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > con fragmentación externa un pedido espera aunque la memoria libre total alcance 0ms
✓ tests/modelos/Simulador.comparacionPolíticas.test.ts > Comparación de políticas con la misma carga > al liberarse el bloque que separa dos huecos, la coalescencia elimina la fragmentación al que esperaba 8ms

Test Files  1 passed (1)
Tests      14 passed (14)
Start at   21:46:56
Duration   1.24s (transform 64%, import 33%, tests 2%, worker 1%)

PS C:\paradigmas\AE2\simulador-de-memoria>

```

Figura 11. Ejecución de la comparación de políticas (`npx vitest run tests/modelos/Simulador.comparacionPolíticas.test.ts --reporter=verbose`): 14 pruebas aprobadas.

16. Análisis de la fragmentación externa

La fragmentación externa mide cuánto de la memoria libre está "partida" en huecos que no sirven para un pedido grande:

$$\text{Fragmentación externa (\%)} = (1 - \text{mayor hueco contiguo} / \text{memoria libre total}) \times 100$$

Si toda la memoria libre está en un solo bloque, la fragmentación es 0 %. Si no hay memoria libre, se define como 0 %. Cuanto más se acerca a 100 %, más repartida está la memoria libre en huecos chicos.

Situaciones observadas

- **Caso 7 del Anexo I:** con huecos no contiguos de 100 y 300 KB, la memoria libre es 400 KB y el mayor hueco 300 KB: $(1 - 300/400) \times 100 = 25 \%$.
- **Pedido que espera aunque la memoria alcanza:** en el escenario A con First-Fit quedan 30 KB libres, pero repartidos en huecos de 10 y 20 KB (fragmentación 33,3 %). Un pedido de 30 KB queda Esperando Memoria. Este es el efecto real de la fragmentación externa: la memoria existe, pero no es contigua.
- **La liberación reduce la fragmentación.** En la última prueba de la comparación, cuando termina el proceso de 10 KB que separaba los dos huecos, la coalescencia une los tres tramos en un bloque de 40 KB: la fragmentación baja de 33,3 % a 0 % y en el tick siguiente se admite el proceso que esperaba.

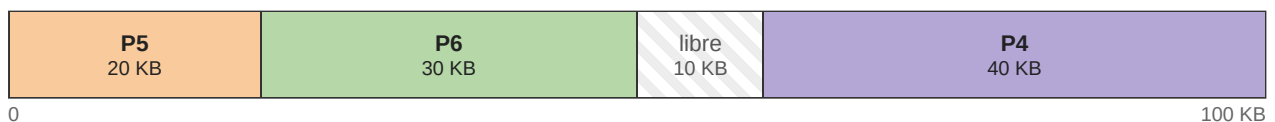
Antes: huecos de 10 y 20 KB separados por P2 → fragmentación 33,3 %



P2 termina: la coalescencia une los huecos en 40 KB → fragmentación 0 %



Tick siguiente: P6 (30 KB) por fin entra



Cómo influyen la liberación y la reorganización. La coalescencia solo une huecos que ya son vecinos: no mueve procesos. Por eso reduce la fragmentación cuando se libera un bloque entre dos huecos, pero no puede eliminar la fragmentación entre huecos separados por procesos vivos. Para eso haría falta **compactación** (mover los procesos para juntar toda la memoria libre), que la consigna excluye y que en un sistema real tiene un costo alto. La política elegida también influye: Best-Fit tiende a generar restos chicos, y Worst-Fit restos grandes.

17. Conclusiones

- Se construyó una biblioteca que cumple los diez requerimientos funcionales, verificada con **316 pruebas** que pasan en su totalidad y con **100 % de cobertura** del código de producción.
- Repartir el comportamiento en objetos con una responsabilidad cada uno hizo que el `Simulador` quedara como un coordinador simple, y que cada parte se pudiera probar por separado.
- El doble encapsulamiento (atributos privados, sin setters, transiciones validadas y vistas inmutables) permitió que el estado nunca quedara inválido: cualquier camino no permitido lanza un error, y los tests lo comprueban.
- La herencia se usó solo donde había una relación "es un" real (las políticas); en el resto, la composición resultó más flexible.
- El análisis comparativo mostró que **ninguna política de asignación es la mejor siempre**: depende de cómo coinciden los pedidos con los huecos.
- La fragmentación externa puede dejar procesos esperando aunque la memoria libre total alcance. La coalescencia la reduce, pero no la elimina sin compactación.
- Escribir las pruebas junto con cada clase, con commits chicos, permitió detectar errores temprano y dejar documentado en la bitácora cada decisión y cada corrección.

18. Bibliografía y fuentes

- Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). *Operating System Concepts* (10.^a ed.). Wiley. Capítulos de planificación de CPU y memoria principal.
- Tanenbaum, A. S. y Bos, H. (2015). *Modern Operating Systems* (4.^a ed.). Pearson. Capítulos de procesos y administración de memoria.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9.^a ed.). Pearson.
- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Template Method, Factory).
- Martin, R. C. (2017). *Clean Architecture*. Prentice Hall. (Principios SOLID).
- Microsoft. *TypeScript Handbook*. <https://www.typescriptlang.org/docs/>
- Vitest. *Guía y documentación de cobertura*. <https://vitest.dev/guide/coverage>
- JGraph. *draw.io: documentación de diagramas UML*. <https://www.drawio.com/doc/>
- Universidad de la Cuenca del Plata (2026). *Escenario AE2: Simulador de procesos y memoria con POO y Consigna de la Actividad de Evaluación 2 — Sistemas Operativos*.

19. Anexo: cómo reproducir los resultados

```
git clone https://github.com/valenlacour/simulador-de-memoria.git
cd simulador-de-memoria
git checkout v1.0
npm ci
npm run typecheck      # verificación de tipos
npm test               # 316 pruebas
npm run test:coverage  # pruebas + cobertura (reporte en coverage/index.html)
npx vitest run -t "Round Robin"  # un caso puntual
```

Modificaciones en vivo para la defensa. El quantum, la memoria total y la política se cambian en la configuración del simulador dentro de un test, por ejemplo `new Simulador(new ConfiguracionSimulador(2048, 3, NombrePolitica.WORST_FIT))`. Un evento de E/S se agrega con `simulador.definirEventoES(pid, cpuConsumida, duracion)`. Para provocar la liberación de un proceso en un tick determinado se lo registra con la CPU necesaria para que termine en ese tick; al terminar, su memoria se libera y se fusiona con los huecos vecinos.